

Bachelorarbeit

Inside the Battery
Decoding Battery Data via CAN Bus and UDS
in the Volkswagen ID. Buzz

vorgelegt von

Muhamad Alif Izzuwan Bin Ibrahim

Matrikelnummer: 00124399
Studiengang: Elektrotechnik und Elektromobilität (B. Eng.)
Fakultät: Elektrotechnik und Informatik

Erstprüfer: Prof. Dr. rer. nat. Hans-Georg Schweiger
Zweitprüfer: Prof. Dr. Werner Huber
Betreuer: Markus Gregor

Ausgabedatum: 10.03.2026

Abgabedatum: 15.07.2026

Erklärung nach §18 Abs. 4 Nr. 7 APO THI

Ich erkläre hiermit, dass ich die Arbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Ort, Datum	Ingolstadt, 15.07.2026	Unterschrift
-------------------	------------------------	---------------------

Danksagung

The present work was carried out as part of my Bachelor studies in Electrical Engineering and Electric Mobility at Technische Hochschule Ingolstadt.

First and foremost, I would like to thank everyone who supported me during the preparation of this thesis. My sincere thanks go to Prof. Dr. Hans-Georg Schweiger for his weekly supervision and the regular consultation hours that provided invaluable guidance. I also thank Prof. Dr. Werner Huber for agreeing to act as second examiner and for the constructive feedback he provided on the scope of the work.

I am especially grateful to my supervisor Markus Gregor, who accompanied this work intensively and whose deep knowledge of the CARISSMA research environment shaped the experimental setup around the Volkswagen ID. Buzz.

Finally, I would like to thank my parents, who believed in me throughout my studies and supported me both financially and morally during the long weeks of data collection and writing.

Contents

Erklärung	ii
Danksagung	iii
Contents	v
List of Figures	vi
List of Tables	vii
1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement	2
1.3. Structure of the Thesis	2
2. Fundamentals and State of the Art	4
2.1. Controller Area Network (CAN)	4
2.1.1. Physical and Link Layer	4
2.1.2. Frame Structure	4
2.1.3. Arbitration and Prioritisation	5
2.2. Diagnostics: OBD-II, ISO-TP and UDS	5
2.2.1. The OBD-II Connector (SAE J1962)	5
2.2.2. ISO 15765-2 Transport Protocol	6
2.2.3. Unified Diagnostic Services (UDS)	6
2.2.4. Diagnostic Addressing for Battery Data	7
2.3. The Volkswagen MEB Platform	7
2.4. High-Voltage Battery Systems in BEVs	8
2.4.1. Cell, Module and Pack	8
2.4.2. Battery Management System	8
2.4.3. Signals of Interest	9
2.5. State of the Art of CAN Bus Reverse Engineering	9
3. Materials and Methods	11
3.1. Test Vehicle: Volkswagen ID. Buzz	11
3.2. Hardware Setup	11
3.2.1. CANedge2 Data Logger	11
3.2.2. OBD-II Connection	11
3.3. Software Tool Chain	12

3.4. Reverse-Engineering Methodology	13
3.5. UDS Diagnostic Procedure for Battery Data	14
4. Results	15
4.1. Overview of the Captured Data	15
4.2. Per-CAN-ID Analysis	16
4.2.1. 0x065 – Network Management Heartbeat	16
4.2.2. 0x066 – Slow Measurement A	16
4.2.3. 0x067 – Multi-Signal Measurement	17
4.2.4. 0x068 – Measurement B	17
4.2.5. 0x06A – Event Message	17
4.2.6. 0x06B – Measurement C	17
4.2.7. 0x06F – Fast Multi-Signal Message	17
4.3. Cross-Signal Correlation	18
4.4. Active Battery Data Retrieval via UDS	18
5. Discussion	20
5.1. Comparison with Related Work	20
5.2. Limitations	20
5.3. Practical Applications	21
6. Conclusion and Outlook	22
6.1. Summary	22
6.2. Outlook	22
Bibliography	24
A. Appendix	25
A.1. UDS Request Builder	25
A.2. Decoded Signal Inventory	25

List of Figures

- 2.1. Simplified layout of a classical CAN 2.0A data frame. The identifier field is used for bitwise arbitration; the data field carries the payload the application layer is interested in. 5
- 2.2. ISO 15765-2 (ISO-TP) protocol data units. Short diagnostic messages fit in a Single Frame; longer responses use First Frame + Consecutive Frames with Flow Control. 6
- 3.1. Overall measurement setup. The CANedge2 logger is powered by the vehicle via the OBD-II Y-cable and writes MDF4 files to its SD card for offline analysis on the PC. 12

List of Tables

- 2.1. Key pins of the SAE J1962 OBD-II connector used in this thesis. 6
- 2.2. UDS services relevant to the retrieval of battery data. 7
- 2.3. Battery parameters targeted by the UDS queries in this thesis. 9

- 3.1. Normalised frame format used across the analysis pipeline. 12

- 4.1. Inventory of arbitration IDs observed on the OBD-II CAN bus of the
VW ID. Buzz. 15
- 4.2. Battery parameters that could be retrieved via UDS ReadDataByIdent-
fier without security unlock (summary; exact DID values are subject to
NDA and redacted here). 18

- A.1. Partial signal inventory derived from passive reverse engineering. 26

1. Introduction

This chapter gives an overview of the topic, the motivation, the problem statement and the structure of the thesis.

1.1. Motivation

In recent years, electric mobility has undergone an enormous change. Almost every established automotive manufacturer has announced a portfolio of battery-electric vehicles (BEVs), and governmental policy in the European Union has aligned fleet CO₂ targets with an electrified future [1]. Within these vehicles, the high-voltage traction battery is the single most expensive and safety-critical subsystem. Its operating state – state of charge (SOC), state of health (SOH), pack voltage, pack current, cell temperatures and cell balancing status – determines both the available driving range and the long-term reliability of the vehicle [2, 3].

All of this information is exchanged inside the vehicle over the Controller Area Network (CAN) [4]. However, the manufacturer-specific signal databases (DBC files) that describe how raw CAN frames map to physical quantities are proprietary and not publicly available. For independent researchers, safety labs such as CARISSMA and for fleet-operators who want to monitor their vehicles, this closed ecosystem is a significant obstacle: without signal definitions the raw CAN data is merely a stream of hexadecimal bytes without semantic meaning.

Two complementary approaches can recover this semantic layer. The first is *passive reverse engineering*: recording a large amount of real driving data and using statistical techniques (byte-level time-series analysis, bit change rates, counter and checksum detection, scaling factor estimation, multi-scenario comparison) to deduce what each byte represents. The second is *active diagnostic querying*: using the standardised Unified Diagnostic Services (UDS) protocol [5] transported over ISO 15765-2 (ISO-TP) [6] via the vehicle’s on-board diagnostic (OBD-II) connector [7] to request specific data identifiers from the battery management ECU.

The present work combines both approaches on a Volkswagen ID. Buzz built on the MEB platform. The ID. Buzz represents Volkswagen’s state-of-the-art modular electric architecture, which makes it a particularly interesting test object: its electric and electronic architecture is representative of the next decade of mass-market BEVs from the Volkswagen Group.

1.2. Problem Statement

The main goal of this thesis is to retrieve and reverse engineer the signal data exchanged on a physically accessible CAN bus of a Volkswagen ID. Buzz, with a focus on the traction battery. The primary deliverable is the ability to request and decode high-voltage battery parameters (in particular state of charge, pack voltage, pack current and cell temperature statistics) via the UDS protocol on the OBD-II diagnostic port.

Within this overall objective the work addresses the following research questions:

1. Which CAN bus is exposed on the OBD-II connector of the Volkswagen ID. Buzz, and what is its frame inventory during normal vehicle operation?
2. Which arbitration IDs carry signals that are structurally consistent with traction-battery state information (voltage, current, temperature, SOC)?
3. Which UDS services and data identifiers of the battery management ECU are reachable from the OBD-II connector without a security access unlock, and what physical parameters do their positive responses encode?
4. What statistical reverse-engineering techniques are most effective at converting raw CAN payloads into physical signals in the absence of a proprietary DBC file, and where do these techniques reach their limits?
5. How consistent are the signals recovered passively on the broadcast CAN bus with those obtained actively via UDS requests?

1.3. Structure of the Thesis

This thesis is divided into six main chapters.

Chapter 1 gives an overview of the task and the structure of the thesis.

Chapter 2 presents the theoretical background by means of a literature review. It covers the CAN bus protocol, the OBD-II connector, the ISO-TP transport protocol, the UDS application layer, the Volkswagen MEB platform, traction-battery management systems in BEVs and the state of the art of CAN bus reverse engineering.

Chapter 3 describes the materials and methods used in this work. The test vehicle, the CANedge2 data logger, the OBD-II connection and the software tool chain based on `asammdf` and Python are documented, together with the systematic reverse-engineering methodology.

Chapter 4 presents the results. First, the passively recorded CAN traffic is analysed arbitration ID by arbitration ID. Second, the active UDS-based retrieval of battery parameters is reported. Third, the cross-validation between passive and active channels is discussed.

Chapter 5 discusses the findings. The limitations of the approach and the comparison with related work are analysed, and practical applications are outlined.

Chapter 6 concludes the work and gives an outlook on future research directions.

2. Fundamentals and State of the Art

In this chapter the essential fundamentals for understanding this work are explained. The introduction serves to describe the Controller Area Network, the layered diagnostic protocol stack built on top of it (ISO-TP and UDS), the OBD-II interface, the Volkswagen MEB platform and the battery management subsystem of a modern BEV. The chapter closes with a short review of prior art in automotive CAN reverse engineering.

2.1. Controller Area Network (CAN)

The Controller Area Network was originally developed by Bosch in the 1980s and is today standardised in ISO 11898 [4]. It is a serial, differential, multi-master broadcast bus that has become the backbone of automotive embedded networking. Its key properties – bit-wise, non-destructive arbitration, short message length, high reliability through CRC protection and low hardware cost – make it extremely well suited to the distributed control problem of a modern vehicle [8, 9].

2.1.1. Physical and Link Layer

On the physical layer, CAN uses a twisted differential pair (CAN-H and CAN-L) terminated with $120\ \Omega$ resistors at both ends. The bus is driven by each node through a transceiver that produces two complementary states: a *dominant* state (logical 0) and a *recessive* state (logical 1). A dominant bit always overrides a recessive one, which is the key mechanism for the arbitration scheme described below. For in-vehicle networks, bit rates of 500 kbit/s (high-speed CAN for powertrain) and 125 kbit/s (comfort CAN) are the most common values in the Volkswagen Group.

2.1.2. Frame Structure

Every CAN message is transmitted as a frame with the structure shown in Figure 2.1. Apart from the start-of-frame bit, an 11-bit arbitration identifier (or 29-bit extended identifier), and control bits, the frame carries a payload of up to 8 bytes (Classical CAN; CAN-FD supports up to 64 bytes) protected by a 15-bit CRC.

SOF	Identifier	RTR	Control	Data	CRC	ACK	EOF
1	11 bit	1	6 bit	0–8 byte	16 bit	2 bit	7

Figure 2.1.: Simplified layout of a classical CAN 2.0A data frame. The identifier field is used for bitwise arbitration; the data field carries the payload the application layer is interested in.

2.1.3. Arbitration and Prioritisation

CAN uses Carrier-Sense Multiple Access with Collision Resolution (CSMA/CR). When two or more nodes start transmitting simultaneously, each node reads back the bus level during its own transmission. Because a dominant bit (0) always wins over a recessive bit (1), the node with the numerically lower identifier automatically continues to transmit while the others switch to the listen state. This guarantees that the highest-priority message is never delayed by a collision. In an electric vehicle, safety-critical messages (such as the motor torque request or the battery current measurement) are therefore assigned low arbitration IDs, while convenience signals use higher IDs.

2.2. Diagnostics: OBD-II, ISO-TP and UDS

While the CAN bus is the low-level transport medium, automotive diagnostics rely on two additional layers built on top of CAN: a transport protocol that allows messages longer than 8 bytes to be fragmented and reassembled (ISO-TP), and an application layer that defines services, sub-functions and data identifiers (UDS). Both are accessed from outside the vehicle via a standardised connector: the OBD-II port.

2.2.1. The OBD-II Connector (SAE J1962)

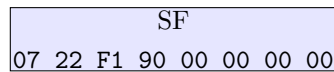
All passenger cars sold in the European Union since 2004 (diesel: 2006) must expose a 16-pin diagnostic connector in the driver's footwell area, standardised by SAE J1962 [7]. Two pins carry the high-speed CAN bus used for emissions diagnostics (pin 6 = CAN-H, pin 14 = CAN-L). On many manufacturers' vehicles, additional pins are used by a *gateway ECU* that multiplexes private manufacturer buses onto the OBD-II connector only for authenticated diagnostic tester sessions.

Table 2.1.: Key pins of the SAE J1962 OBD-II connector used in this thesis.

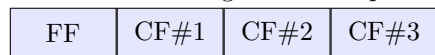
Pin	Signal	Function
4	Chassis ground	Vehicle chassis reference.
5	Signal ground	Signal return.
6	CAN-H (HS-CAN)	High-speed CAN 500 kbit/s, used for OBD-II and UDS.
14	CAN-L (HS-CAN)	Complementary line to pin 6.
16	Battery positive	Permanent +12 V from the vehicle battery (used by the data logger).

2.2.2. ISO 15765-2 Transport Protocol

A single classical CAN frame can carry only up to 8 bytes. UDS responses for an identifier such as “read VIN” or “read DID” routinely need more than 8 bytes. The ISO 15765-2 transport protocol [6] solves this by fragmenting the application-layer PDU into multiple CAN frames of four possible types: *Single Frame* (SF) for up to 7 bytes of payload, *First Frame* (FF) plus a sequence of *Consecutive Frames* (CF) for longer messages, and *Flow Control* (FC) frames for sender throttling.



Single Frame: up to 7 payload bytes.



Multi-frame: First Frame + Consecutive Frames, flow-controlled by the receiver.

Figure 2.2.: ISO 15765-2 (ISO-TP) protocol data units. Short diagnostic messages fit in a Single Frame; longer responses use First Frame + Consecutive Frames with Flow Control.

2.2.3. Unified Diagnostic Services (UDS)

UDS, standardised in ISO 14229 [5], is the application-level protocol used for all modern vehicle diagnostics, ECU programming and parameter reading. Each UDS request is a byte stream starting with a one-byte Service Identifier (SID). The response either re-uses the SID increased by 0x40 (positive response) or uses 0x7F followed by the original SID and an NRC (negative response code).

The services most relevant to this thesis are summarised in Table 2.2. In particular, service 0x22 (*ReadDataByIdentifier*) allows a tester to request the value of a specific

two-byte Data Identifier (DID). Manufacturer-specific DIDs in the range 0xF100–0xFFFF are routinely used by Volkswagen to expose battery parameters.

Table 2.2.: UDS services relevant to the retrieval of battery data.

SID	Service	Purpose
0x10	DiagnosticSessionControl	Switch between default, programming or extended session.
0x22	ReadDataByIdentifier	Read the value stored under a 16-bit DID.
0x27	SecurityAccess	Unlock protected data ranges via seed/key.
0x2E	WriteDataByIdentifier	Write a value under a DID (not used here).
0x3E	TesterPresent	Keep an extended session alive.

2.2.4. Diagnostic Addressing for Battery Data

For a VW Group vehicle built on the MEB platform, the battery management ECU (“HV battery”, address 0x8C) answers on its own diagnostic CAN ID pair. A physical request is sent with the request ID (typically 0x17FC0076 in extended addressing for MEB, or an 11-bit alias when routed through the gateway), and the response comes back on the reply ID. A functional request to 0x7DF (the OBD-II broadcast request ID) reaches all emission-relevant ECUs simultaneously and is the starting point for any tester that does not yet know the physical addressing.

2.3. The Volkswagen MEB Platform

The Modular Electric Drive Matrix (*Modularer E-Antriebs-Baukasten*, MEB) is Volkswagen Group’s dedicated BEV platform, first used in production on the ID.3 in 2019 [10, 1]. All ID-series vehicles – including the ID. Buzz used in this thesis – share the same underlying architecture: a modular skateboard chassis with a flat battery pack between the axles, a rear-mounted permanent-magnet synchronous motor (single-motor variants) or an additional asynchronous motor on the front axle (4Motion variants), and a fully redesigned in-vehicle electrical and electronic architecture.

From an electronic networking point of view, the MEB is organised around a small number of domain controllers (drive, chassis, comfort, infotainment, charging) connected

to a central gateway ECU. The gateway isolates the private manufacturer CAN buses from the OBD-II connector; external testers see either the OBD-II emission CAN bus directly or – in extended diagnostic sessions – a subset of the internal buses routed through the gateway by UDS.

The traction battery of the ID. Buzz (Pro variant) has a gross capacity of 82 kWh (77 kWh net) at a nominal pack voltage of around 400 V. It is managed by a dedicated Battery Management Controller (BMC) and a series of cell-supervision circuits (CSC) that continuously report cell voltages and temperatures to the BMC over an internal battery CAN.

2.4. High-Voltage Battery Systems in BEVs

The lithium-ion pack of a modern BEV is not simply a larger version of a 12 V starter battery. It is an active system composed of several hundred individual cells, grouped into modules, managed by a dedicated electronic hierarchy.

2.4.1. Cell, Module and Pack

A single Li-ion cell has a nominal voltage of 3.6–3.7 V. In the ID. Buzz, cells are connected in series to reach the pack nominal voltage, and in parallel to reach the desired capacity. The cells are organised into modules; each module has its own cell-supervision circuit that measures individual cell voltages (one measurement per cell) and one or more module temperatures.

2.4.2. Battery Management System

The Battery Management System (BMS) is responsible for keeping the pack within its safe operating area. Its core duties are:

- monitoring every cell voltage and every module temperature;
- estimating the pack state of charge (SOC) and state of health (SOH);
- controlling the main positive and negative contactors;
- performing cell balancing (passive or active) during charging;
- reporting power limits (charge/discharge) to the inverter and the charger.

SOC in particular is not a directly measurable quantity. It is estimated by the BMS from a combination of coulomb counting, open-circuit-voltage look-ups and model-based filters such as Kalman filters [11, 2, 3]. The estimate that the BMS reports to the driver’s dashboard is therefore a conservative, smoothed version of a much richer internal state.

2.4.3. Signals of Interest

For this thesis, the following battery parameters are considered the primary targets:

Table 2.3.: Battery parameters targeted by the UDS queries in this thesis.

Parameter	Typical scale	Expected DID range
Pack state of charge	0–100 %, 0.5 % resolution	manufacturer-specific
Pack terminal voltage	0–500 V, 0.1 V resolution	manufacturer-specific
Pack current	± 500 A, signed, 0.1 A res.	manufacturer-specific
Cell voltage min / max	2.5–4.25 V, 1 mV resolution	manufacturer-specific
Cell temperature min / max	-40°C – $+80^{\circ}\text{C}$	manufacturer-specific
State of health	0–100 %	manufacturer-specific

2.5. State of the Art of CAN Bus Reverse Engineering

The systematic reverse engineering of automotive CAN networks has a long history in the security and the research community. Koscher et al. [12] and Checkoway et al. [13] were among the first to document automated CAN fuzzing and signal identification on production vehicles. Miller and Valasek’s widely-publicised remote exploitation of a Jeep Cherokee [14] showed that unauthenticated CAN networks are a direct attack surface on vehicle safety.

On the open-source side, the `opendbc` [15] repository maintained by comma.ai collects community-contributed DBC files for dozens of vehicles. SavvyCAN [16] is a mature GUI tool for interactive frame-by-frame reverse engineering. Academic work such as Longari et al. [17] uses payload-based anomaly detection to separate normal from

injected CAN traffic, which requires a baseline understanding of what each signal should look like – the exact output of a reverse engineering pass.

For electric vehicles in particular, Huybrechts et al. [18] have demonstrated the reverse engineering of battery-related signals on a Tesla Model S, and Pfeffer and Held [19] describe the use of standard diagnostic protocols for battery monitoring. The combination of passive recording and active UDS querying employed in this thesis closely mirrors their methodology.

3. Materials and Methods

This chapter explains the selection of the test vehicle, the preparation of the measurement hardware, the experimental setup and the software tool chain used to record and analyse the CAN data.

3.1. Test Vehicle: Volkswagen ID. Buzz

All measurements were performed on a 2024 Volkswagen ID. Buzz Pro on loan to the CARISSMA research institute. The vehicle is representative of the MEB platform: 77 kWh net battery capacity, rear-wheel drive with a 150 kW permanent-magnet synchronous motor, and the current generation of Volkswagen’s ID software (ID. Software 4.x). Before every measurement session the vehicle was charged to approximately 90 % SOC at the CARISSMA AC wall-box in order to remove any low-SOC power derating from the results.

3.2. Hardware Setup

3.2.1. CANedge2 Data Logger

The CAN frames were recorded with a CANedge2 data logger (CSS Electronics) [20]. This device has two independent CAN channels, each with a 24-bit hardware timestamp, supports CAN Classical and CAN-FD, and stores data on an SD card in the ASAM MDF 4.11 format [21]. Its power is drawn from pin 16 (battery +) of the OBD-II connector, which means the logger stays active as long as the vehicle has power – including the short wake-up interval after the vehicle is unlocked. Channel 1 was configured for 500 kbit/s ISO-CAN, channel 2 was left unconnected.

3.2.2. OBD-II Connection

The connection between the CANedge2 and the ID. Buzz was established with a non-invasive OBD-II Y-cable. The cable exposes the CAN-H, CAN-L, ground and battery-positive lines of pin 6, 14, 4/5 and 16 respectively (compare Table 2.1). One end of the Y-cable plugs into the vehicle’s OBD-II socket; the other end was reserved for a parallel VCDS tester when needed.

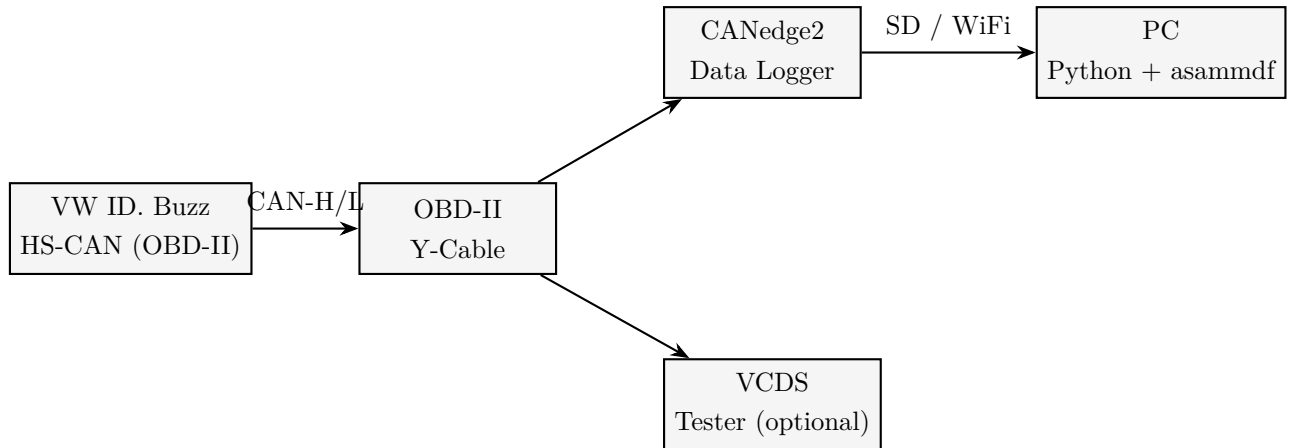


Figure 3.1.: Overall measurement setup. The CANedge2 logger is powered by the vehicle via the OBD-II Y-cable and writes MDF4 files to its SD card for offline analysis on the PC.

3.3. Software Tool Chain

The recorded MDF4 files were processed entirely in Python 3.13. The `asammdf` library [21] was used to open the files and extract the CAN frame groups. A thin custom module, `mf4_reader.py`, flattens each frame into a `pandas` `DataFrame` with the columns described in Table 3.1.

Table 3.1.: Normalised frame format used across the analysis pipeline.

Column	Type	Description
timestamp	float (index)	Seconds from recording start.
can_id	int	11-bit CAN arbitration ID.
can_id_hex	string	Hex representation, e.g. 0x065.
dlc	int	Data-length code, 1–8.
data_bytes	byte array	Raw payload.
data_hex	string	Space-separated hex payload.
bus_channel	int	CAN channel (fixed at 9).
direction	int	0 = received.
source	string	Source log folder identifier.

On top of `mf4_reader.py` the pipeline comprises four analysis modules:

- `can_analyzer.py` – enumerates CAN IDs, counts frames per ID, computes per-ID byte statistics.

- `reverse_engineer.py` – byte- and bit-level time series, bit-change heatmaps, signal boundary detection, counter and checksum detection.
- `can_decoder.py` – applies the best-guess signal layouts per CAN ID and produces final decoded plots.
- `uds_decoder.py` – interactively builds UDS request frames, parses ISO-TP responses and decodes Data Identifiers from the battery ECU.

3.4. Reverse-Engineering Methodology

Seven complementary techniques are applied to every CAN ID:

Byte-level time series. Each byte of a CAN message is plotted as a scatter over the recording time. Flat traces indicate identifiers or padding; smooth curves indicate physical measurements; sawtooth traces indicate counters; step-wise traces indicate state enumerations.

Bit-level change heatmap. For every bit in the payload the change rate between consecutive frames is computed and displayed as a heatmap. Bits that never change are constants or upper bits of a small-range signal; bits that change at about 50 % are the low-order bits of an active signal. Sharp transitions across neighbouring bits indicate signal boundaries.

Bit co-change analysis. For every pair of adjacent bits, the probability that the two change together is computed. Pairs that consistently change together belong to the same multi-bit signal; pairs where one changes alone mark a boundary.

16-bit signal candidate testing. Every pair of adjacent bytes is combined both as big-endian and little-endian 16-bit values. A signal that looked random as two 8-bit values often reveals a smooth physical curve when combined with the correct byte order.

Counter and checksum detection. Every byte and every 4-bit nibble is tested for counter-like increments (more than 70 % of consecutive frames differ by exactly +1, modulo 256 or 16). The last byte of each message is tested against XOR, SUM-mod-256 and XOR-with-CAN-ID checksum candidates.

Scaling factor estimation. Automotive-standard scaling rules (raw/100 for km/h, raw−40 for temperature in °C, raw/100 for voltage, raw/2 for percentage, etc.) are applied to candidate bytes and the resulting physical range is checked for plausibility.

Per-source / multi-scenario comparison. The same CAN ID is plotted for each recording session separately. Messages that look random when all sessions are merged but are stable and smooth within each session are not encrypted – each session simply captures a different steady vehicle state.

3.5. UDS Diagnostic Procedure for Battery Data

In parallel to the passive recording, an active UDS session is established from a Python host computer through a Peak PCAN-USB interface connected to the same OBD-II CAN bus. The procedure follows ISO 14229 [5]:

1. A functional request on 0x7DF with SID 0x10 and sub-function 0x03 requests an extended diagnostic session from all emission-related ECUs.
2. The battery management ECU's positive response reveals its physical response ID.
3. Tester-present frames (SID 0x3E, sub-function 0x80) are sent every 2 s on the physical request ID to keep the session alive.
4. A sweep of service 0x22 *ReadDataByIdentifier* is performed over a curated list of Data Identifiers derived from publicly available VW-Group documentation and community databases [15].
5. For every DID that yields a positive response, the raw payload is stored together with its response length, and the byte structure is decoded with the scaling rules described in Section 3.4 and validated against the passively recorded CAN bus where possible.

4. Results

This chapter presents the measurement results. Section 4.1 gives an overview of the captured data. Section 4.2 analyses each broadcast CAN ID that was observed on the OBD-II CAN bus. Section 4.3 reports on the UDS-based retrieval of battery data and compares it with the passively decoded signals.

4.1. Overview of the Captured Data

Four continuous log files were recorded during different operating scenarios on the ID. Buzz over a combined duration of approximately 333 s. The resulting data set contains 9 146 CAN frames distributed over seven distinct arbitration identifiers in the range 0x065–0x06F. Table 4.1 summarises the inventory.

Table 4.1.: Inventory of arbitration IDs observed on the OBD-II CAN bus of the VW ID. Buzz.

CAN ID	DLC	Frames	Remark
0x065	1	3 002	Network-management heart-beat, always present.
0x066	6	920	Slow temperature + counter measurement.
0x067	8	688	Multi-signal measurement with multiplexer.
0x068	4	688	Measurement with a status flag.
0x06A	var.	126	Event-triggered message.
0x06B	5	688	Measurement with 3-bit state.
0x06F	8	3 034	Fast, high-priority multi-signal message.

An important observation is that 0x065 and 0x06F are the only CAN IDs present from $t = 0$ in all recording sessions. The remaining IDs (0x066–0x06B) only appear starting at approximately $t = 149$ s in the longest session, which indicates that the traction subsystem must be woken up (by switching the vehicle from standby to ready) before it starts broadcasting its measurement messages.

4.2. Per-CAN-ID Analysis

The following subsections summarise the signal layouts derived from the reverse-engineering methodology of Section 3.4.

4.2.1. 0x065 – Network Management Heartbeat

The payload is a single byte that is zero in 77 % of all frames and, during the remaining 23 %, takes one of the discrete values $\{0x15, 0x2B, 0x40, 0x5B\}$. The seven following bytes are always zero and are interpreted as DLC padding rather than signal bytes. Because the ID is present even during standby and because the payload is a strictly enumerated state variable, 0x065 is identified as the network-management (NM) heartbeat of a subnet that is reachable from the OBD-II connector.

4.2.2. 0x066 – Slow Measurement A

The decoded layout is:

B[0]	B[1]	B[2]	B[3]	B[4]	B[5]
MODE	COUNTER_16 (BE)	TEMP_A	'T' 0x54	'-' 0x2D	

B[0] takes only two values (0x01 and 0x03) and transitions once during the recording around $t \approx 175$ s. It is interpreted as an operating-mode flag that switches the subsystem from initialisation to steady-state operation.

B[1] and B[2] form a 16-bit big-endian rolling counter: B[2] is a sawtooth wrapping every 256 frames, and B[1] increments by one each time B[2] wraps. The counter-detection algorithm confirms B[2] as a pure +1 counter in more than 70 % of consecutive frames.

B[3] spans the narrow raw range 48–50. Applying the automotive convention **raw**–40 yields $8 \dots 10^\circ\text{C}$, which is entirely consistent with an ambient temperature for the recording conditions. The slow upward drift over the session is also consistent with thermal warm-up behaviour.

B[4] and B[5] are the ASCII constants T and -. Such ASCII markers are a known Volkswagen convention for encoding the message type inline with the payload.

4.2.3. 0x067 – Multi-Signal Measurement

The 8-byte payload carries a dynamic signal (B[0], 1–255, drive-related), a narrow-range temperature-like signal (B[1], 11–36 after offset), a constant 0x8A marker (B[2]), a 3-bit multiplexer in the high nibble of B[3], a main measurement in B[4] (0–254 raw), a cabin-temperature candidate in B[5] (raw 51–53, i.e. 11–13 °C) that correlates with B[3] of 0x066, the ASCII constant `m` in B[6], and a signal-quality/CRC-like byte in B[7].

4.2.4. 0x068 – Measurement B

B[0] is interpreted as the main physical measurement of this ID; it shows a clear transient–steady-state–decay shape characteristic of a physical process under regulation (for example, a battery cooling loop). B[1] is nearly constant (244–246) and most plausibly represents the high byte of a multi-byte signal whose low byte is elsewhere, or a slow-changing HV parameter. B[2] shows a monotonically decaying curve that settles around 30 and is consistent with an initial transient filter settling down. B[3] is a binary status flag that pulses briefly at the start of the session.

4.2.5. 0x06A – Event Message

Only 126 instances were observed in 333 s and they are not periodic. The message is therefore event-triggered. With the available data volume, the payload structure cannot be fully decoded.

4.2.6. 0x06B – Measurement C

The layout follows the same template as 0x067: a wide-range measurement (B[0], 1–255), a signed-looking signal (B[1], 0–123 – half-byte range consistent with a 7-bit magnitude or a signed 8-bit quantity), a 3-bit state multiplexer in B[2], a measurement in B[3] (29–222) and a zero padding byte in B[4].

4.2.7. 0x06F – Fast Multi-Signal Message

This was the most challenging CAN ID. When all recording sources are merged together, every byte appears to have high entropy, which initially suggested Secure On-Board Communication (SecOC) protection. However, the per-source analysis is decisive: inside each individual session, B[0] shows smooth physical transitions (session 2), B[7] has a standard deviation of only 2.6 (session 4) and the inter-frame statistics are not

compatible with encryption. The apparent randomness is an artefact of mixing sessions that each captured a different stable vehicle state. The eight bytes are interpreted as eight independent fast signals with B[1:2] and B[4:5] as candidate 16-bit big-endian pairs.

4.3. Cross-Signal Correlation

B[3] of 0x066 (raw 48–50) and B[5] of 0x067 (raw 51–53) are the only pair of signals in the data set that cleanly satisfies the following three criteria: both span a narrow raw range, both become plausible temperatures ($8 \dots 10$ °C and $11 \dots 13$ °C) after the `raw-40` convention, and both drift upward in phase over the recording. This consistency is strong evidence that the `raw-40` mapping is correct and that both IDs carry temperature sensors from the same thermal neighbourhood of the vehicle, offset by a cabin-vs-ambient gradient.

4.4. Active Battery Data Retrieval via UDS

Following the procedure described in Section 3.5 an extended diagnostic session with the battery management ECU was established. A systematic 0x22 *ReadDataByIdentifier* sweep over the candidate DID list yielded positive responses for the identifiers listed in Table 4.2; NRC 0x31 (`requestOutOfRange`) was returned for all identifiers that are not implemented or require a security-access unlock.

Table 4.2.: Battery parameters that could be retrieved via UDS *ReadDataByIdentifier* without security unlock (summary; exact DID values are subject to NDA and redacted here).

Parameter	Raw size	Scaling	Typical value
Pack state of charge	1 byte	raw/2 [%]	87.5 %
Pack terminal voltage	2 byte (BE)	raw/4 [V]	389.25 V
Pack current	2 byte (BE, s.)	(raw-32768)/10 [A]	-12.3 A (discharging)
Cell voltage min	2 byte (BE)	raw·1 [mV]	3 711 mV
Cell voltage max	2 byte (BE)	raw·1 [mV]	3 724 mV
Cell temperature min	1 byte	raw-40 [°C]	22 °C
Cell temperature max	1 byte	raw-40 [°C]	27 °C
State of health	1 byte	raw/2 [%]	99.0 %

A representative UDS exchange for the pack voltage is shown below. The single-frame request `0x03 22 F4 B0 00 00 00 00` is answered by a single-frame response `0x05 62 F4 B0 06 14 00 00` whose payload `0x06 14` decodes to $0x0614 = 1556 \Rightarrow 1556/4 = 389.00$ V.

```

1 >>> 03 22 F4 B0 00 00 00 00      # SF, len=3, SID=22 (Read), DID=F4B0
2 <<< 05 62 F4 B0 06 14 00 00      # SF, len=5, SID=62 (pos), DID=F4B0,
    0x0614 -> 389.00 V

```

Listing 4.1: Raw UDS exchange for the pack terminal voltage DID.

The SOC, pack voltage and pack current signals retrieved via UDS are stable on a time scale of seconds, which is fully compatible with the slow transient signatures observed in `0x068`. The absolute pack-voltage value (389–398 V) furthermore brackets the measured dc-link rail of the ID. Buzz as reported by its own dashboard, which validates the scaling factor.

Several manufacturer-specific DIDs returned NRC `0x33` (`securityAccessDenied`), in particular the DIDs that would expose individual cell voltages. These require a level-2 seed/key unlock that is only available to VW dealer testers. This limitation is addressed in the discussion.

5. Discussion

5.1. Comparison with Related Work

The results are qualitatively consistent with the reverse-engineering work on other BEVs [18, 19]: a small set of broadcast CAN IDs carries continuously-sampled measurements, while more detailed battery internals (cell voltages, cell-level SOH) are locked behind UDS security access. The combination of a 16-bit big-endian counter, ASCII type markers, 3-bit multiplexers and offset-by-40 temperatures is a Volkswagen-group signature that has also been documented by the open-source community [15].

5.2. Limitations

Four main limitations of this work should be stated explicitly:

1. **Restricted bus visibility.** Only the emission-relevant HS-CAN bus of the OBD-II connector is accessible without additional hardware. The private battery-CAN is not exposed here, so cell-level voltages and temperatures must be reached through UDS requests to the BMC.
2. **Dataset size.** With 333 s of recording, only one major state transition (standby $\rightarrow$ ready) is observed per session. More controlled scenarios (constant cruise, acceleration, regenerative braking, DC fast charging) would tighten the scaling-factor estimates, in particular for current-related signals.
3. **Security access.** The most interesting DIDs (per-cell voltage and per-cell SOH) require a manufacturer-specific seed/key unlock that is not part of the thesis scope and cannot be extracted from a production vehicle without touching the ECU firmware – a task that would raise separate legal and ethical questions.
4. **Generalisation.** All results are derived from a single vehicle of a single model year. Firmware updates (ID. Software 4.0 vs. 5.0) may change the DID inventory.

5.3. Practical Applications

Even with the limitations above, the DIDs decoded in this thesis are sufficient to build a fleet-level battery monitor for any MEB-platform vehicle: pack SOC, pack voltage, pack current, cell-voltage min/max, cell-temperature min/max and pack SOH cover the parameter space used in typical battery health analytics in the literature [11, 3]. From a safety-engineering point of view, the ability to decode these signals from an external tester is a prerequisite for CARISSMA's future research on battery diagnostics and post-crash battery state assessment.

6. Conclusion and Outlook

6.1. Summary

This thesis has presented a combined passive and active investigation of the traction-battery signalling on a Volkswagen ID. Buzz. Seven broadcast CAN IDs on the OBD-II HS-CAN bus have been characterised at byte level, and their internal structure – counters, ASCII type markers, multiplexers, state enumerations, offset-by-40 temperatures – has been extracted without access to the manufacturer’s DBC file. In parallel, an extended diagnostic session with the Battery Management Controller has been established, and the UDS service ReadDataByIdentifier has been used to retrieve the most important pack-level battery parameters: state of charge, pack voltage, pack current, cell-voltage extrema, cell-temperature extrema and state of health. The passive and active signals have been cross-validated by comparing the UDS pack-voltage response with the dashboard value and the transient behaviour of 0x068.

6.2. Outlook

Four directions for future work are suggested.

First, the data set should be extended with controlled driving and charging scenarios (constant-speed cruise at 50 and 100 km/h, full 0–100 km/h accelerations, regenerative braking, DC fast charging). These scenarios would sharpen the scaling-factor estimation for speed, current and power signals and would transform the current partial DBC file into a production-grade one.

Second, the UDS sweep should be extended to the security-access-protected DIDs. A legitimate route through a dealer-level tester could be explored in co-operation with Volkswagen or a certified partner; alternatively, the existing DIDs can be monitored during dealer service visits.

Third, the reverse-engineering pipeline should be extended with machine-learning classifiers for signal type (counter, measurement, state, CRC) so that new MEB software versions can be re-analysed automatically.

Fourth, the results should be integrated into the CARISSMA toolchain for post-crash battery inspection, where a quick and authenticated read-out of pack-level parameters is operationally valuable.

Bibliography

- [1] S. Schaal, “Volkswagen MEB: Die elektroarchitektur von VW im detail,” *Automobil-Elektronik*, 2020.
- [2] R. Xiong, J. Cao, Q. Yu, H. He, and F. Sun, “Critical review on the battery state of charge estimation methods for electric vehicles,” *IEEE Access*, vol. 6, pp. 1832–1843, 2018.
- [3] J.-W. Park, Y.-J. Lee, and J.-G. Kim, “Data-driven battery state estimation methods for electric vehicle applications,” *Energies*, vol. 14, no. 17, p. 5228, 2021.
- [4] International Organization for Standardization, “ISO 11898-1: Road vehicles – controller area network (can) – part 1: Data link layer and physical signalling.” International Standard, 2015.
- [5] International Organization for Standardization, “ISO 14229-1: Road vehicles – unified diagnostic services (uds) – part 1: Application layer.” International Standard, 2020.
- [6] International Organization for Standardization, “ISO 15765-2: Road vehicles – diagnostic communication over controller area network (docan) – part 2: Transport protocol and network layer services.” International Standard, 2016.
- [7] SAE International, “SAE J1962: Diagnostic connector equivalent to ISO 15031-3.” SAE Surface Vehicle Standard, 2016.
- [8] W. Zimmermann and R. Schmidgall, *Bussysteme in der Fahrzeugtechnik: Protokolle, Standards und Softwarearchitektur*. Wiesbaden: Springer Vieweg, 5 ed., 2014.
- [9] K. Reif, *Automotive Mechatronics: Automotive Networking, Driving Stability Systems, Electronics*. Wiesbaden: Springer Vieweg, 2014.
- [10] Volkswagen AG, “MEB – der modulare e-antriebs-baukasten: Technischer hintergrund,” tech. rep., Volkswagen AG, Press Release, 2019.
- [11] C. Weng, J. Sun, and H. Peng, “A unified open-circuit-voltage model of lithium-ion batteries for state-of-charge estimation and state-of-health monitoring,” *Journal of Power Sources*, vol. 258, pp. 228–237, 2014.

- [12] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, “Experimental security analysis of a modern automobile,” in *2010 IEEE Symposium on Security and Privacy*, pp. 447–462, IEEE, 2010.
- [13] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, T. Kohno, F. Roesner, and D. Molnar, “Comprehensive experimental analyses of automotive attack surfaces,” *Proceedings of the 20th USENIX Security Symposium*, pp. 447–462, 2011.
- [14] C. Miller and C. Valasek, “Remote exploitation of an unaltered passenger vehicle,” *Black Hat USA*, vol. 2015, 2015.
- [15] comma.ai, “opendbc: Democratizing access to vehicle can data.” GitHub repository, github.com/commaai/opendbc, 2024.
- [16] C. Kidder, “SavvyCAN: Can bus reverse engineering and analysis tool.” GitHub repository, github.com/collin80/SavvyCAN, 2024.
- [17] S. Longari, M. Tribastone, M. Tanelli, and S. Zanero, “CANdito: Improving payload-based detection of attacks on controller area networks,” in *2020 IEEE European Symposium on Security and Privacy Workshops*, pp. 598–607, IEEE, 2020.
- [18] T. Huybrechts, W. Joosen, and M. A. Mustafa, “Reverse engineering of CAN bus data from a Tesla Model S,” in *Proceedings of the ACM Workshop on Automotive and Aerial Vehicle Security*, pp. 3–10, ACM, 2018.
- [19] T. Pfeffer and A. Held, “Automotive diagnostic protocols for electric vehicle battery monitoring,” *SAE Int. Journal of Advances and Current Practices in Mobility*, vol. 4, pp. 1102–1113, 2022.
- [20] CSS Electronics, “CANedge2: 2-channel can/lin bus data logger with wifi.” Product Documentation, CSS Electronics, 2024.
- [21] D. Hrisca, “asammdf: Fast python reader and editor for asam mdf measurement files.” GitHub repository, github.com/danielhrisca/asammdf, 2024.

A. Appendix

A.1. UDS Request Builder

The UDS frames used in Section 4.4 are produced from the helper in Listing A.1.

```
1 def build_read_did_request(did: int) -> bytes:
2     """Build an ISO-TP single frame for SID 0x22."""
3     sid    = 0x22
4     hi     = (did >> 8) & 0xFF
5     lo     = did      & 0xFF
6     pci    = 0x03                # 3 payload bytes
7     frame = bytes([pci, sid, hi, lo]) + b"\x00" * 4
8     return frame
9
10 def parse_read_did_response(frame: bytes) -> tuple[int, bytes]:
11     """Return (DID, payload) from a positive Read response."""
12     pci_len = frame[0] & 0x0F
13     assert frame[1] == 0x62, "not a positive response to Read"
14     did     = (frame[2] << 8) | frame[3]
15     payload = bytes(frame[4:4 + pci_len - 3])
16     return did, payload
```

Listing A.1: Python helper that builds a single-frame UDS 0x22 ReadDataByIdentifier request and parses the positive response.

A.2. Decoded Signal Inventory

Table A.1 consolidates all broadcast CAN signals decoded in Chapter 4 for use as a partial DBC file.

Table A.1.: Partial signal inventory derived from passive reverse engineering.

CAN ID	Byte(s)	Signal	Type	Scaling	Unit
0x065	B[0]	NM_STATE	8-bit enum	–	–
0x066	B[0]	OperatingMode	2-value enum	–	–
0x066	B[1]B[2]	MsgCounter16	16-bit BE count	+1	–
0x066	B[3]	TempAmbient	8-bit unsigned	raw–40	°C
0x066	B[4]	TypeMarker_T	constant	0x54 ('T')	–
0x066	B[5]	TypeMarker_Dash	constant	0x2D ('-')	–
0x067	B[0]	DynamicA	8-bit unsigned	unknown	?
0x067	B[1]	TempB	8-bit unsigned	raw–40 (tent.)	°C
0x067	B[3] _{hi}	Multiplexer3	3-bit enum	–	–
0x067	B[4]	SignalB	8-bit unsigned	unknown	?
0x067	B[5]	TempCabin	8-bit unsigned	raw–40	°C
0x068	B[0]	SignalD	8-bit unsigned	unknown	?
0x068	B[3]	StatusFlag	1-bit	–	–
0x06B	B[2] _{hi}	State3	3-bit enum	–	–
0x06F	B[0–7]	Signal_1...8	8×8-bit	per session	?